

ΠΑΡΑΛΛΗΛΑ ΣΥΣΤΗΜΑΤΑ

ΑΣΚΗΣΗ-2Β



AM: 161173

Ονοματεπώνυμο: Βεσέλ Νταλίπαϊ

Μέρος 1

Το πρόγραμμα ξεκινάει ανοίγοντας το αρχείο args και διαβάζει από εκεί πρώτα το N (μέγεθος του πίνακα A) και μετά το threads. Αν το αρχείο δεν ανοίξει ή αν οι τιμές δεν είναι έγκυρες, το πρόγραμμα τερματίζει. Υπάρχει έλεγχος ώστε να μην συνεχίσει όταν $threads * threads > N$ για να αποφύγουμε να έχουμε παραπάνω threads per block από ό,τι στήλες ή γραμμές.

Στη συνέχεια δεσμεύεται μνήμη στον host για τον πίνακα A και διαβάζονται από το αρχείο όλα τα στοιχεία του πίνακα A. Αφού ολοκληρωθεί η ανάγνωση, το πρόγραμμα συνεχίζει με τη CUDA πλευρά.

Υπολογίζουμε το blocksPer ώστε να έχουμε αρκετά blocks για να καλυφθούν όλα τα στοιχεία του πίνακα σε κάθε διάσταση. Μετά ορίζουμε ότι κάθε block θα έχει διάσταση threads x threads. Τέλος, φτιάχνουμε το 2D grid που καλύπτει όλον τον πίνακα. Το πρόγραμμα στο τέλος θα τυπώνει πόσα threads έχει κάθε block, πόσα blocks έχει συνολικά το grid και πόσα threads θα εκτελεστούν συνολικά.

Έπειτα δημιουργούνται CUDA events για τη χρονομέτρηση. Υπάρχει ένα ζευγάρι events για τον συνολικό χρόνο, ένα ζευγάρι για τον υπολογισμό των sum/max/mean και ένα ζευγάρι για το βήμα κατασκευής του πίνακα B ή C.

Στη device μνήμη δεσμεύονται οι πίνακες και οι μεταβλητές που χρειάζονται. Δεσμεύεται dA για τον πίνακα A, dOut για τον πίνακα εξόδου, καθώς και dSum και dMax που θα κρατήσουν αντίστοιχα το συνολικό άθροισμα και το μέγιστο στοιχείο του A. Μετά γίνεται αντιγραφή του hA (ο πίνακας A στον host) στο dA και αρχικοποιούνται οι dSum και dMax στη GPU με τιμές 0.0 και 0 αντίστοιχα ώστε να μην έχουμε σκουπίδια.

Ακολουθεί το kernel `sum_max`, όπου κάθε thread αντιστοιχεί σε ένα στοιχείο του πίνακα A με δείκτες που προκύπτουν από `blockIdx` και `threadIdx`. Κάθε thread, εφόσον βρίσκεται εντός ορίων του πίνακα, διαβάζει την τιμή του στοιχείου και ενημερώνει το συνολικό άθροισμα με `atomicAdd` και το μέγιστο με `atomicMax`. Μετά αντιγράφονται στον host οι τιμές του αθροίσματος και του μέγιστου και υπολογίζεται η μέση τιμή `mean`.

Στη συνέχεια ο κώδικας ελέγχει ποιος πίνακας θα κατασκευαστεί. Αν ισχύει $hMax > N * mean$ τότε ενεργοποιείται η κατασκευή του πίνακα B, αλλιώς κατασκευάζεται ο πίνακας C. Για το αποτέλεσμα δεσμεύεται στον host ένας πίνακας `hOut`, ο οποίος θα κρατήσει είτε τον B είτε τον C μετά την αντιγραφή από τη GPU.

Αν επιλεγεί ο B, καλείται το kernel `build_B`. Σε αυτό κάθε thread υπολογίζει το αντίστοιχο στοιχείο του B με βάση τη μέση τιμή και το `max`. Υπάρχει έλεγχος του `max` με το 0 ώστε να μην γίνει διαίρεση με το μηδέν. Μετά γίνεται `cudaMemcpy` του αποτελέσματος και βρίσκουμε στον host το ελάχιστο στοιχείο του B με ένα απλό πέρασμα σε όλα τα στοιχεία (δεν κατάφερα να το κάνω στην `gpu`).

Αν επιλεγεί ο C, καλείται το kernel `build_C`. Σε αυτό κάθε thread υπολογίζει τους δείκτες `jplus` και `jminus` για τη δεξιά και αριστερή γειτονική στήλη. Όταν `column+1==N` χρησιμοποιεί στήλη 0 και όταν `column==0` χρησιμοποιεί στήλη N-1. Έπειτα υπολογίζει το στοιχείο του πίνακα C προσθέτοντας την τιμή στη θέση `(row, column)` και τις δύο διπλανές της στην ίδια γραμμή, τα διαιρεί δια 3 και γράφει το αποτέλεσμα. Μετά το αποτέλεσμα αντιγράφεται στον host.

Μετά από κάθε κομμάτι εκτέλεσης ο κώδικας ελέγχει σφάλματα CUDA με `cudaGetLastError` και χρησιμοποιεί `HANDLE_ERROR` στις CUDA κλήσεις ώστε να τερματίζει σε περίπτωση αποτυχίας. Οι χρόνοι μετρώνται με τα CUDA events και στο τέλος τυπώνονται ο χρόνος για το βήμα

sum/max/mean, ο χρόνος για την κατασκευή του B ή C και ο συνολικός χρόνος εκτέλεσης.

Υπάρχει η συνάρτηση print_matrix που τυπώνει τον πίνακα B ή C, αλλά το έχουμε αφήσει σε σχόλια ώστε για μεγάλα N να μην παράγεται τεράστιο output.

Τέλος, το πρόγραμμα αποδεσμεύει τη device μνήμη και καταστρέφει τα CUDA events.

Ενδεικτικά τρεξίματα

```
C:\Users\vesel\Desktop\Cuda>nvcc -allow-unsupported-compiler -gencode arch=compute_86,code=sm_86 -o ASK1 ASK1.cu
ASK1.cu
tmpxft_00008878_00000000-7_ASK1.cudafe1.cpp

C:\Users\vesel\Desktop\Cuda>.\ASK1.exe
Matrix size: N = 8
Total elements: 64
Threads: 2
Threads per block: 4
Total blocks: 16
Total threads in grid: 64
Mean of matrix A: 5.281250
Maximum element of matrix A: 43
Matrix build: B

B matrix:
 0.0763  0.0298  0.0298  0.0065 -0.0400 -0.0400 -0.0632  0.0531
-0.0632 -0.0632 -0.0632 -0.0167  0.0996  0.0298  0.0531 -0.0865
 0.0996 -0.0167  0.0298 -0.0632 -0.0167  0.1228 -0.0167 -0.0167
 0.0298  0.0996 -0.0167  0.0996 -0.0865 -0.0865 -0.8772  0.1228
 0.0996 -0.0632  0.0531  0.0298 -0.0632 -0.0400  0.0065  0.0065
-0.0865  0.0531  0.0065  0.0996 -0.0632  0.0763  0.0065  0.0996
-0.0865  0.0298 -0.0865  0.0996  0.1228  0.0996 -0.0865  0.0996
-0.0632  0.0531  0.0298  0.0763  0.0996  0.0298  0.0065  0.0065

Minimum value of matrix B (amin(B)): -0.877180
Sum,max and mean time: 0.212 ms
Build B: 0.059 ms
Total execution time: 0.297 ms
```

```
C:\Users\vesel\Desktop\Cuda>.\ASK1.exe
```

```
Matrix size: N = 8
```

```
Total elements: 64
```

```
Threads: 2
```

```
Threads per block: 4
```

```
Total blocks: 16
```

```
Total threads in grid: 64
```

```
Mean of matrix A: 11.062500
```

```
Maximum element of matrix A: 12
```

```
Matrix build: C
```

```
C matrix:
```

11.0000	11.0000	11.0000	11.3333	11.0000	11.3333	11.0000	11.3333
11.6667	11.3333	10.6667	10.0000	10.6667	11.0000	11.3333	11.3333
11.0000	11.3333	11.6667	11.3333	11.0000	11.0000	11.0000	10.6667
10.6667	11.3333	11.6667	11.6667	11.6667	11.3333	10.6667	10.0000
11.6667	11.3333	11.3333	11.0000	11.0000	10.3333	11.0000	11.3333
11.3333	10.6667	11.0000	10.6667	11.0000	10.6667	11.0000	10.6667
10.6667	10.0000	10.6667	10.6667	11.3333	10.6667	11.3333	10.6667
11.0000	11.3333	11.6667	11.3333	11.6667	11.6667	11.3333	11.0000

```
Sum,max and mean time: 0.213 ms
```

```
Build C: 0.064 ms
```

```
Total execution time: 0.314 ms
```

```
C:\Users\vesel\Desktop\Cuda>.\ASK1.exe
```

```
Matrix size: N = 2048
```

```
Total elements: 4194304
```

```
Threads: 16
```

```
Threads per block: 256
```

```
Total blocks: 16384
```

```
Total threads in grid: 4194304
```

```
Mean of matrix A: 4.495547
```

```
Maximum element of matrix A: 9220
```

```
Matrix build: B
```

```
Sum,max and mean time: 7.561 ms
```

```
Build B: 5.163 ms
```

```
Total execution time: 12.757 ms
```

```
C:\Users\vesel\Desktop\Cuda>.\ASK1.exe
```

```
Matrix size: N = 2048
```

```
Total elements: 4194304
```

```
Threads: 16
```

```
Threads per block: 256
```

```
Total blocks: 16384
```

```
Total threads in grid: 4194304
```

```
Mean of matrix A: 11.095897
```

```
Maximum element of matrix A: 12
```

```
Matrix build: C
```

```
Sum,max and mean time: 7.564 ms
```

```
Build C: 2.287 ms
```

```
Total execution time: 9.885 ms
```

Μέρος 2

Το πρόγραμμα ξεκινάει ανοίγοντας το αρχείο `args` και διαβάζει από εκεί πρώτα το `N` (διάσταση του πίνακα `A`, δηλαδή `A` είναι $N \times N$) και μετά το `threads`. Αν το αρχείο δεν ανοίξει ή αν οι τιμές δεν είναι έγκυρες, το πρόγραμμα τερματίζει. Υπάρχει έλεγχος ώστε να μην συνεχίσει όταν $threads \times threads > N$, για να αποφύγουμε να ορίσουμε block με υπερβολικά πολλά threads σε σχέση με το `N`.

Στη συνέχεια δεσμεύεται μνήμη στον host για τον πίνακα `A` και διαβάζονται από το αρχείο όλα τα στοιχεία του πίνακα `A`. Αφού ολοκληρωθεί η ανάγνωση, το αρχείο κλείνει και το πρόγραμμα συνεχίζει με τη CUDA πλευρά.

Μετά δεσμεύεται μνήμη στον host για τους πίνακες `hSums` και `hC`. Ο πίνακας `hSums` αρχικοποιείται με μηδενικά γιατί θα κρατήσει τα αθροίσματα των στηλών, και ο πίνακας `hC` επίσης μηδενίζεται πριν σταλεί στη GPU.

Υπολογίζουμε το `blocksPer` ώστε να έχουμε αρκετά blocks για να καλυφθούν όλα τα στοιχεία του πίνακα σε κάθε διάσταση. Μετά ορίζουμε ότι κάθε block θα έχει διάσταση `threads x threads` και φτιάχνουμε ένα 2D grid που καλύπτει όλον τον πίνακα. Το πρόγραμμα στο τέλος τυπώνει πόσα threads έχει κάθε block, πόσα blocks έχει συνολικά το grid και πόσα threads θα εκτελεστούν συνολικά.

Έπειτα δημιουργούνται CUDA events για τη χρονομέτρηση. Υπάρχει ένα ζευγάρι events για τον συνολικό χρόνο, ένα ζευγάρι για το βήμα υπολογισμού των `sums` και `means`, ένα ζευγάρι για το βήμα αφαίρεσης των μέσων τιμών (δημιουργία του `B`) και ένα ζευγάρι για το βήμα υπολογισμού του άνω τριγωνικού του πίνακα συνδιακύμανσης (πίνακας `C`).

Στη device μνήμη δεσμεύονται οι πίνακες που χρειάζονται. Δεσμεύεται dA για τον πίνακα A, dSums για τα αθροίσματα στηλών, dMeans για τις μέσες τιμές στηλών, dB για τον πίνακα (A-mean ανά στήλη) και dC. Μετά γίνεται αντιγραφή του hA στο dA, καθώς και αντιγραφή των μηδενισμένων hSums και hC στα dSums και dC ώστε οι υπολογισμοί να ξεκινήσουν από γνωστές τιμές και όχι από σκουπίδια μνήμης.

Αρχικά εκτελείται το kernel sum_columns, όπου κάθε thread αντιστοιχεί σε ένα στοιχείο του A με δείκτες που προκύπτουν από blockIdx και threadIdx. Κάθε thread, εφόσον βρίσκεται εντός ορίων, προσθέτει το στοιχείο του A στο άθροισμα της αντίστοιχης στήλης με atomicAdd ώστε στο τέλος το sums[column] να ισούται με το άθροισμα όλων των στοιχείων της στήλης.

Αμέσως μετά εκτελείται το kernel compute_means, το οποίο είναι 1D και κάθε thread αντιστοιχεί σε μία στήλη. Για κάθε column, υπολογίζει τη μέση τιμή της συγκεκριμένης στήλης.

Στη συνέχεια εκτελείται το kernel subtract_mean, όπου κάθε thread αντιστοιχεί ξανά σε ένα στοιχείο του πίνακα και υπολογίζει τον πίνακα B ως $B[\text{row} * N + \text{column}] = A[\text{row} * N + \text{column}] - \text{means}[\text{column}]$. Με αυτόν τον τρόπο, από κάθε στοιχείο αφαιρείται η μέση τιμή της στήλης του.

Έπειτα εκτελείται το kernel conv_upper, όπου κάθε thread αντιστοιχεί σε ένα στοιχείο $C[\text{row}][\text{column}]$ αλλά υπολογίζει μόνο το άνω τριγωνικό μέρος του πίνακα. Για κάθε ζευγάρι (row, column) υπολογίζει ένα άθροισμα τύπου dot product και αποθηκεύει το αποτέλεσμα. Έτσι υπολογίζεται η συνδιακύμανση ανάμεσα στις στήλες row και column του B, και επειδή ο πίνακας είναι συμμετρικός δεν χρειάζεται να υπολογιστεί και το κάτω τριγωνικό.

Μετά την εκτέλεση του `con_upper` αντιγράφουμε το αποτέλεσμα πίσω στον `host`.

Έχουμε σε σχόλια ένα κομμάτι κώδικα που συμπληρώνει το κάτω τριγωνικό μέρος καθρεφτίζοντας το άνω τριγωνικό και μετά τυπώνει ολόκληρο τον πίνακα `C`, αλλά το έχουμε αφήσει σε σχόλια για να μην έχουμε τεράστιο `output` σε μεγάλα `N`.

Μετά από κάθε κομμάτι εκτέλεσης ο κώδικας ελέγχει σφάλματα CUDA με `cudaGetLastError` και χρησιμοποιεί `HANDLE_ERROR` στις CUDA κλήσεις ώστε να τερματίζει σε περίπτωση αποτυχίας. Οι χρόνοι μετρώνται με τα `CUDA events` και στο τέλος τυπώνονται ο χρόνος για το βήμα υπολογισμού των `means`, ο χρόνος για την αφαίρεση των μέσων τιμών, ο χρόνος για τον υπολογισμό της συνδιακύμανσης και ο συνολικός χρόνος εκτέλεσης.

Τέλος, το πρόγραμμα αποδεσμεύει τη `device` μνήμη και καταστρέφει τα `CUDA events`.

Ενδεικτικά τρεξίματα

```
C:\Users\vesel\Desktop\Cuda>nvcc -allow-unsupported-compiler -gencode arch=compute_86,code=sm_86 -o ASK2 ASK2.cu
ASK2.cu
tmpxft_0000070c_00000000-7_ASK2.cudafe1.cpp

C:\Users\vesel\Desktop\Cuda>.\ASK2.exe
Matrix size: N = 16
Total elements: 256
Threads: 4
Threads per block: 16
Total blocks: 16
Total threads in grid: 256
Column means computation: 0.215 ms
Subtracting means : 0.027 ms
Covariance upper computation: 0.067 ms
Total time: 0.342 ms

Covariance matrix C (N=16):
13360.000  3560.000  -140.000  -5140.000  -6440.000  -6640.000  -2140.000  5960.000  13360.000  3560.000  -3040.000  -5140.000  -6940.000  -5540.000  160.000  9460.000
3560.000  11260.000  6310.000  -1190.000  -3740.000  -6440.000  -5690.000  -1340.000  3560.000  11260.000  2160.000  -1190.000  -5490.000  -6590.000  -4640.000  910.000
-140.000  6310.000  10735.000  1985.000  -1190.000  -5140.000  -6265.000  -3790.000  -140.000  6310.000  5960.000  1985.000  -3565.000  -5915.000  -5840.000  -2165.000
-5140.000  -1190.000  1985.000  10735.000  6310.000  -140.000  -5015.000  -6290.000  -5140.000  -1190.000  10735.000  10735.000  2685.000  -2165.000  -5840.000  -5915.000
-6440.000  -3740.000  -1190.000  6310.000  11260.000  3560.000  -3190.000  -6340.000  -6440.000  -3740.000  2160.000  6310.000  7010.000  910.000  -4640.000  -6590.000
-6640.000  -6440.000  -5140.000  -140.000  3560.000  13360.000  2860.000  -4040.000  -6640.000  -6440.000  -3040.000  -140.000  8060.000  9460.000  160.000  -5540.000
-2140.000  5960.000  -6265.000  -5015.000  -3190.000  2860.000  16735.000  4210.000  -2140.000  -5690.000  -6040.000  -5015.000  -565.000  7085.000  12160.000  835.000
5960.000  -1340.000  -3790.000  -6290.000  -6340.000  -4040.000  4210.000  16060.000  5960.000  -1340.000  -5440.000  -6290.000  -5590.000  -1690.000  7760.000  10810.000
13360.000  3560.000  -140.000  -5140.000  -6440.000  -6640.000  -2140.000  5960.000  13360.000  3560.000  -3040.000  -5140.000  -6940.000  -5540.000  160.000  9460.000
3560.000  11260.000  6310.000  -1190.000  -3740.000  -6440.000  -5690.000  -1340.000  3560.000  11260.000  2160.000  -1190.000  -5490.000  -6590.000  -4640.000  910.000
-3040.000  2160.000  5960.000  5960.000  2160.000  -3040.000  -6040.000  -5440.000  -3040.000  2160.000  10560.000  5960.000  -840.000  -4440.000  -6240.000  -4440.000
-5140.000  -1190.000  1985.000  10735.000  6310.000  -140.000  -5015.000  -6290.000  -5140.000  -1190.000  5960.000  10735.000  2685.000  -2165.000  -5840.000  -5915.000
-6940.000  -5490.000  -3565.000  2685.000  7010.000  8060.000  -565.000  -6940.000  -5490.000  -840.000  2685.000  12135.000  4785.000  -2640.000  -6465.000  -6465.000
-5540.000  -6590.000  -5915.000  -2165.000  910.000  9460.000  7085.000  -1690.000  -5540.000  -6590.000  -4440.000  -2165.000  4785.000  14935.000  3760.000  -3815.000
160.000  -4640.000  -5840.000  -5840.000  -4640.000  160.000  12160.000  7760.000  160.000  -4640.000  -5840.000  -2640.000  3760.000  16960.000  3760.000  3760.000
9460.000  910.000  -2165.000  -5915.000  -6590.000  -5540.000  835.000  10810.000  9460.000  910.000  -4440.000  -5915.000  -6465.000  -3815.000  3760.000  14935.000
```

```
C:\Users\vesel\Desktop\Cuda>nvcc -allow-unsupported-compiler -gencode arch=compute_86,code=sm_86 -o ASK2 ASK2.cu
ASK2.cu
tmpxft_000002e04_00000000-7_ASK2.cudafe1.cpp

C:\Users\vesel\Desktop\Cuda>.\ASK2.exe
Matrix size: N = 256
Total elements: 65536
Threads: 16
Threads per block: 256
Total blocks: 256
Total threads in grid: 65536
Column means computation: 0.192 ms
Subtracting means : 0.036 ms
Covariance upper computation: 0.101 ms
Total time: 0.364 ms
```

```
C:\Users\vesel\Desktop\Cuda>nvcc -allow-unsupported-compiler -gencode arch=compute_86,code=sm_86 -o ASK2 ASK2.cu
ASK2.cu
tmpxft_0000106c_00000000-7_ASK2.cudafe1.cpp

C:\Users\vesel\Desktop\Cuda>.\ASK2.exe
Matrix size: N = 1024
Total elements: 1048576
Threads: 32
Threads per block: 1024
Total blocks: 1024
Total threads in grid: 1048576
Column means computation: 0.189 ms
Subtracting means : 0.038 ms
Covariance upper computation: 1.248 ms
Total time: 1.510 ms
```

Για τον σκοπό των ασκήσεων φτιάξαμε το generator.c το οποίο φτιάχνει το αρχείο args, γράφει το N, τα threads, και μετά έναν τυχαίο πίνακα A (N×N). Τον φτιάχνει έτσι ώστε περίπου 50/50 να βγει B ή C στο μέρος 1.

Το πρόγραμμα έτρεξε σε Windows 11 με επεξεργαστή AMD Ryzen 7 7700 8-Core Processor, 3801 Mhz, 8 Core(s), 16 Logical Processor(s) και GPU NVIDIA GeForce RTX 3070.

Compile κάνουμε με:

```
nvcc -allow-unsupported-compiler -gencode  
arch=compute_86,code=sm_86 -o <filename> <filename>.cu
```